

High-Level Design (HLD)

Professional-Skepticism Engine — Multi-Agent Audit Investigation Crew

Project	Professional-Skepticism Engine	Document	High-Level Design (HLD)
Version	1.2	Status	Reviewed — aligned with PRD v1.2
Owner	Athithiyan A · Aravinth Jay	Date	2026-06-26
Pattern	Multi-agent (debate + verify)	Stack	LangGraph · Claude · SQLite · RAG

v1.2 changes. Adds the **CSV intake & rule pre-filter** as a first-class component (§3.2, §4) and a **per-agent input/output summary** (§6) that the LLD and Sample Output document expand into concrete, worked contracts across two cases (auto-clear + escalated).

1. Introduction

This HLD describes the system-level design of the Professional-Skepticism Engine: its architecture, major components, data flow, technology choices, integrations, and cross-cutting concerns. It is the bridge between the PRD (what & why) and the LLD (how, in detail).

1.1 Scope

In scope: the CSV intake pipeline, the multi-agent investigation pipeline, grounding services (RAG + external APIs), human-in-the-loop, persistence, and the reporting/audit layer for the 5-day MVP. **Out of scope:** production ERP connectors, multi-tenant infra, autonomous ledger posting.

2. System Overview

A general-ledger extract (CSV) is ingested and pre-filtered into cases. For each flagged transaction (a “case”), a crew of role-specialised Claude agents assembles evidence, runs an adversarial Challenger/Defender debate, adjudicates a risk verdict, verifies it is grounded, and — for uncertain or material cases — pauses for a human. State and the debate transcript are persisted; every decision is audited.

3. Architecture

3.1 Architectural style

- **Deterministic intake, then orchestrated reasoning** — a cheap rule pre-filter gates which rows reach the expensive crew.
- **Orchestrated multi-agent graph** — LangGraph StateGraph owns control flow and durable state.
- **Role-separated agents** — each agent is an independent Claude call with one objective and a typed I/O contract.
- **Adversarial + verification loop** — independence of Challenger / Defender / Verifier is the core mechanism.
- **Grounding-before-judgement** — RAG + external APIs build evidence before any reasoning.
- **Human-in-the-loop via durable interrupt** — confidence-gated pause/resume.

3.2 Component view

Component	Responsibility	Key tech
-----------	----------------	----------

Intake pipeline	Ingest GL CSV, normalize rows, run rule pre-filter, create cases for flagged rows only	pandas/csv · Pydantic
Orchestrator	Graph control flow, routing, bounded re-runs, checkpointing	LangGraph StateGraph
Agent layer	6 role-specialised Claude agents (supervisor, evidence, challenger, defender, adjudicator, verifier)	langchain-anthropic
Grounding services	Policy RAG (vector DB) + external-API connectors with cache	Chroma/FAISS · httpx
State & persistence	Durable state + debate transcript + immutable audit log	SQLite checkpointer + tables
HITL / review	Confidence-gated interrupt; reviewer decision + signature	LangGraph interrupt/ resume
Presentation	Case list, debate-transcript viewer, review queue, dashboard	Next.js / React (or Streamlit)

4. End-to-End Data Flow

#	Stage	Data in → out
0a	Ingest	GL CSV row → typed Transaction
0b	Pre-filter	Transaction → flags[]; flagged → case · clean → dropped (cleared-at-intake)
1	Plan	case → plan + routing
2	Evidence	case → evidence{} (RAG + history + APIs) + citations
3	Debate	case+evidence → challenger_view, defender_view
4	Adjudicate	evidence+views → adjudication{risk·conf·disposition}
5	Verify	verification{passed·issues}; loop to adjudicator if failed (max 2 rounds → escalate)
6	Gate	routing: auto-clear / reviewer / partner review
7	Human review	bundle → review{action·signature}
8	Report+Audit	all → case file + audit_history[]

5. Technology Stack

Layer	Technology
Intake	pandas / csv + Pydantic v2 (typed Transaction + rule pre-filter)
Orchestration	LangGraph (StateGraph + SQLite checkpointer, interrupt/resume)
LLM	Anthropic Claude — Sonnet for reasoning agents, Haiku for report generation
Schemas	Pydantic v2
RAG	Chroma / FAISS + embeddings (hybrid search + re-rank)

HTTP / connectors	httpx (timeout + cache + retry)
Persistence	SQLite (checkpoints, cases, reviews, audit_log)
UI	Next.js / React (or Streamlit)

6. Per-Agent Interface Summary (I/O)

Each agent is a stateless Claude call with a typed contract. The concrete Pydantic schemas are in LLD §3; fully worked input/output for two cases (one auto-cleared, one escalated) is in the Sample Output document.

Agent / node	Input	Output (typed)
Intake pre-filter	CSV row	Transaction + flags[] (or dropped)
Supervisor	case	plan{steps, specialists, max_rounds, materiality, escalation}
Evidence	case + plan	Evidence{policy[], history{}, external{}, citations[], api_metadata[]}
Challenger	case + evidence	ChallengerView{position, severity, claims[], red_flags[]}
Defender	case + evidence	DefenderView{position, arguments[], conditions[], confidence_in_legitimacy}
Adjudicator	evidence + both views	Adjudication{risk_level, confidence, disposition, rationale, citations[]}
Verifier	adjudication + evidence	Verification{passed, checked/grounded/ungrounded, issues[], action}
Confidence gate	adjudication + verification + amount	routing{decision, reason}
Human review	full bundle + transcript	ReviewEvent{reviewer_id, action, comment, signature, timestamp}
Report / Audit	full state	CaseFile (MD/HTML/JSON) + audit_history[] (hash-chained)

7. Integration / External Interfaces

Interface	Direction	Purpose	Scope
GL CSV file	in	Source of transactions for intake	MVP
Anthropic Claude	out	Every agent (debate/adjudicate/verify)	MVP
Frankfurter / exchangerate.host	out	Historical FX evidence	MVP
Google Shopping / Open Food Facts	out	Price-reference evidence	Future
GST / GSTN verify	out	Vendor / tax validity	Future
UI	in/out	Upload, review, resume	MVP

8. Deployment View

- **MVP:** single container (Docker) running the intake + orchestrator + UI, with a local SQLite file and a local Chroma index. One Anthropic API key from an environment secret.

- **Future:** split UI / worker services, managed Postgres checkpointer, a message queue for fan-out, and per-tenant isolation — the agent logic is unchanged.

```
[ User ]--HTTP-->[ UI ]--upload CSV-->[ Intake pre-filter ]--cases-->[ Orchestrator (LangGraph) ]
                                     |--> Claude API (agents)
                                     |--> Vector DB (policy RAG)
                                     |--> External APIs (FX/price) [+cache]
                                     |--> SQLite (checkpoints, audit)
[ Reviewer ]--approve/reject-->[ UI ]--resume(thread_id)-->[ Orchestrator ]
```

9. Cross-Cutting Concerns

Concern	Approach
Reliability	Checkpointed state + transcript; bounded re-runs; API retry + cache fallback
Security	Keys in secret store; PII not logged in clear; least privilege; SoD on approvals
Observability	Per-agent traces (tokens/latency/cost) + run metrics; LangSmith optional
Scalability	Stateless agent calls; cheap intake pre-filter caps crew volume; future queue + fan-out
Cost	Rules pre-filter; debate-round caps; evidence cache; model routing per role
Auditability	Immutable hash-chained audit log incl. full debate transcript

10. Key Design Decisions

Decision	Rationale
Deterministic CSV pre-filter before the crew	Keeps the expensive multi-agent debate off the ~95% of clean rows; controls cost & latency
Multi-agent debate (not single prompt)	Independent challenge/defence reduces false positives & bias; provable via RAGAS
Separate Verifier agent	A model can't reliably check itself; independent grounding gate catches hallucination
LangGraph + checkpointer	Durable, resumable human-in-the-loop and bounded re-run loops
RAG for policy	Company-specific, defensible verdicts; policy change = data change
Model routing per role	Cost/quality balance; deterministic temps for judge/verify, higher for advocates

Appendix A — Architecture Diagram

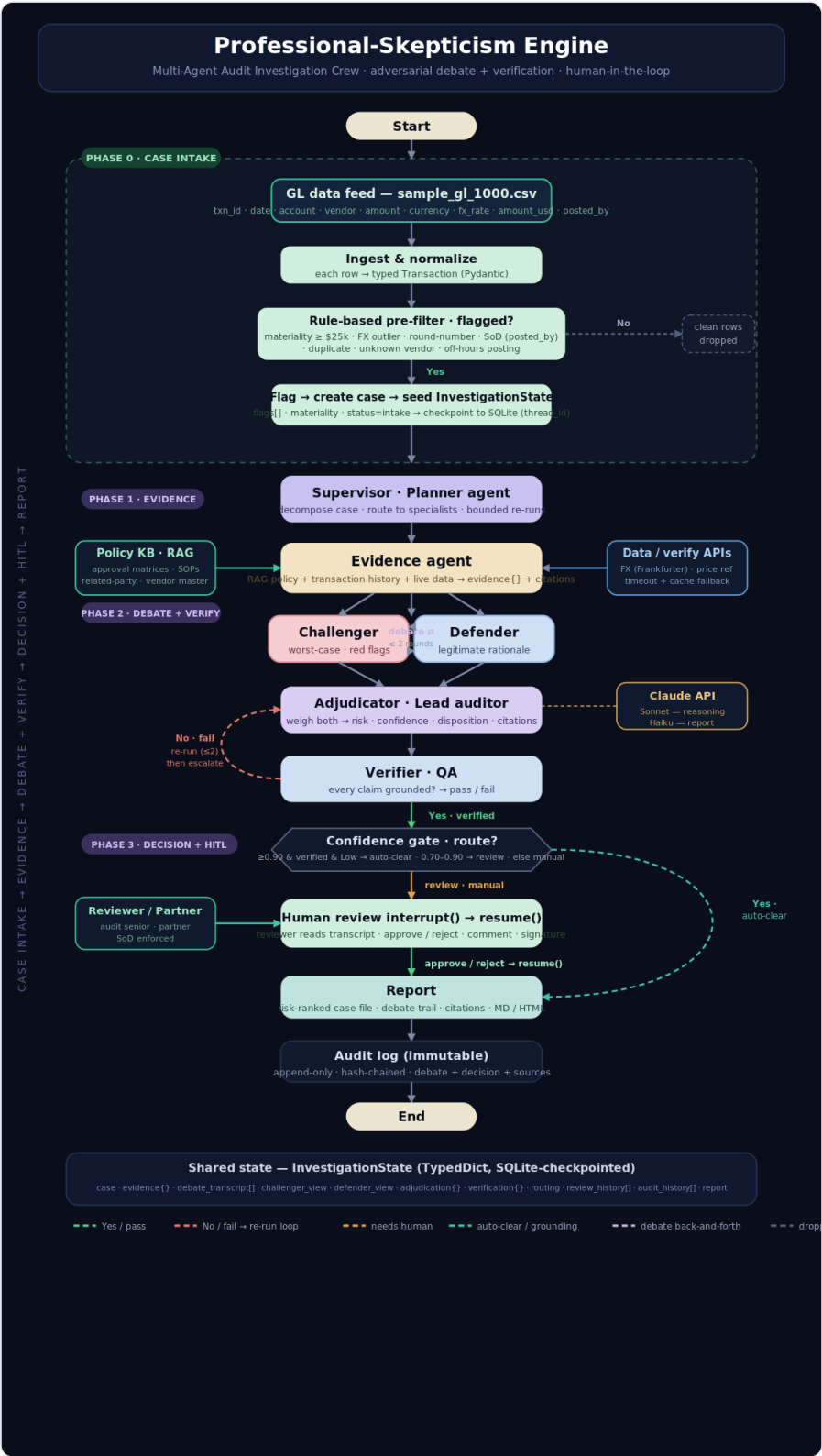


Figure 1 — System architecture with the Phase-0 CSV intake pipeline, the adversarial crew, the verify re-run loop, the confidence gate's yes/no routing, and the human-in-the-loop interrupt.